



■ تقرير مراجعة معمارية وفنية DEEP-DIVE REVIEW

منصة CyberGuard AI مراجعة البنية والكود

تقييم عميق لمستودع Cybersecurity-AI-Agent مع خارطة تطوير
ومقترح منظومة توليد التقارير

وثيقة مرجعية شخصية تُقيّم بنية الكود وأمان التنفيذ وقيمة التعلم في منصة الأمن السيبراني متعددة الوكلاء: تحدد نقاط القوة القابلة للبناء عليها، وترصد فرص التحسين مرتبة بالأولوية مع أمثلة كود موثقة من المستودع ذاته، ثم تفصل خطة تكامل مكثبات توليد التقارير (PDF/DOCX) داخل وحدة ReportBuilder لدعم التعلم والتفاعل بشكل أفضل.

3

مزودو ذكاء اصطناعي

10

وحدات واجهة أمامية

14

نقطة نهاية للـ API

+8,150

سطر مصدري تمت

مراجعتهم

github.com/MOT1209/Cybersecurity-AI-Agent

REPOSITORY

أغسطس 2026 — مراجعة للإصدار fa11d85 (فرع main)

DATE

البنية والكود + منظومة توليد التقارير (تكامل داخلي)

FOCUS

مرجعية تطوير شخصية — اقتراحات دون تعديل الكود

AUDIENCE

الملخص التنفيذي

منصة CyberGuard AI عمل تعليمي طموح ومكتمل الملامح: بواجهة ثنائية اللغة عربية/إنجليزية من الأصل إلى الـ RTL، وطبقة خادم تضم 14 نقطة نهاية، وعشرة أقسام تفاعلية تغطي من تنسيق الوكلاء المتعددين حتى ساحة CTF. الأهم أن المنصة بُنيت بعقلية أمنية صحيحة منذ البداية: تحقق منهجي من المدخلات في كل نقطة نهاية، وتحديد معدل متدرج، وحدود حقن أوامر لنماذج الذكاء الاصطناعي، وبوابة تفويض للأهداف مفهوماً التعليمي قيّم بحد ذاته. هذه الأسس نادرة في مشاريع التعلم، وتستحق أن تُبنى عليها بدل إعادة كتابتها.

في المقابل، تتركز فرصة التحسين الأكبر في ثلاثة محاور معمارية: **أولاً** ملف `server.ts` المونوليثي (1,513 سطرًا) يخلط الميدلوير والمخازن والمنطق والقوالب في وحدة واحدة، **ثانيًا** وجود مسارين متوازيين للاستدعاء الذكي أحدهما يتجاوز طبقة المزودين الموحدة التي بُنيت بعناية، و**ثالثًا** الحالة الكاملة في الذاكرة التي تفقدها المنصة عند كل إعادة تشغيل، ما يمنع تتبع تقدم المتعلم — وهو جوهر هدف «دعم التعلم والتفاعل». على مستوى الواجهة، غياب طبقة تدويل موحدة كلف الكود ثلاثة اصطلاحات ثنائية اللغة متعارضة، واكتُشفت أخطاء وظيفية محددة منها تحدي CTF-02 غير القابل للحل (تم التحقق منه بالفك الفعلي للسلسلة).

بخصوص منظومة التقارير — محور اهتمامك الأساسي — فالوضع الحالي «قالب حرفي» يرجع Markdown ثابتًا من الخادم ويكتفي بتنزيله بصيغة `.md`، بينما يمتلك نموذج البيانات `SecurityFinding` ثراءً مهدورًا (CVSS، الأدلة، كود الترقيع، رقعة الإعداد، خطوات التحصين). التقرير يخص قسمين كاملين لتصميم منظومة تقارير متكاملة عبر مكتبات تكامل داخلي مقارنة تفصيليًا، توصل المنصة من «تنزيل ملف نصي» إلى PDF متجهي عربي بقالب مؤسسي ورسوم توزيع شدة، ثم DOCX قابل للتعديل.

10 / 5

بنود OWASP Top 10 فقط
في قاعدة المعرفة

0

طبقة تخزين دائمة (كل
شيء في الذاكرة)

2

مساران متوازيان
للاستدعاء الذكي

1,513

سطرًا في `server.ts`
المونوليثي

أعلى خمس توصيات حسب الأولوية

الأولوية	التوصية	الأثر المتوقع
P0	إصلاح خطأ ربط CTF بالوكيل (<code>App.tsx:43</code>) وتحدي CTF-02 المعطل	استعادة مسارين تعليميين أساسيين دون أي تعديل معماري
P0	تقسيم <code>server.ts</code> إلى وحدات (<code>routes/services/stores</code>)	قابلية اختبار وصيانة وتمهيد الطبقة الدائمة والتقارير
P1	توحيد الاستدعاء الذكي عبر طبقة LLM المركزية وإلغاء <code>getAIClient()</code> المكرر	تعدد مزودين فعلي في كل المسارات + تقليل 120- سطرًا مركزًا

الأولوية	التوصية	الأثر المتوقع
P1	قاموس تحويل موحد <code>t(key)</code> يستبدل الاصطلاحات الثلاثة	إخماد مئات الثلاثيات المضمنة ومعالجة الحقول العربية-فقط
P2	طبقة تخزين دائمة (SQLite/Prisma) + تحقق خادمي لأعلام CTF	ملف متعلم دائم، تقدم محفوظ، وتحديات غير مختربة من أدوات المطور

ثلاثة مخاطر مفاهيمية تستحق الانتباه: (1) مطابقة النطاق في بوابة الأمان بالسلاسل الجزئية قابلة للتجاوز والإنكار الخاطئ (قسم 3 يوثق أمثلة فعلية)؛ (2) الأعلام والحدود تُشحن ضمن حزمة العميل فيتتحقق التحدي داخل المتصفح؛ (3) بعض الأرقام المعروضة (نسبة نجاح 98.4%، «12 وكيلًا نشطًا»، «عزل 100%») قيم ثابتة في الكود تُقدّم كمؤشرات حية.

SECTION 01

نظرة عامة على المنصة وبنية المستودع

ما هي المنصة؟ CyberGuard AI بيئة تعلم تفاعلية للاختراق الأخلاقي والأمن السيبراني تحاكي — دون تنفيذ فعلي — دورة حياة تقييم أمني كاملة: تنسيق مهمة عبر وكلاء متخصصين (استطلاع، أمن ويب، تحليل ثغرات، معالجة، تقارير...)، بوابة تفويض تفرض حدود النطاق المصرح، مركز استعادة أخطاء بقواطع دائرة، مختبر طرفي لمحاكاة أوامر، صندوق أدوات تشفير وفك تشفير يعمل محليًا في المتصفح، قاعدة معرفة OWASP/MITRE، ساحة تحديات CTF، ومنشئ تقارير اختبار اختراق. الفكرة الجوهرية — تعليم مفاهيم التفويض والعزل والاستعادة «بالتجربة المحاكاة» — سليمة تعليميًا وتمييزها عن كثير من أدوات التعلم التي تعرض محتوى مقروءًا فقط.

حزمة التقنيات: الواجهة 19 React مع 6 Vite و 4 Tailwind CSS و TypeScript صارم، والخادم 4 Express يُشغّل عبر tsx في التطوير ويُجرّم بـ esbuild للإنتاج، مع Vite مدمج كوسيط في وضع التطوير (نمط SPA شائع وفعال). طبقة الذكاء الاصطناعي تلتزم بتجريد `LLMProvider` واحد بثلاثة تطبيقات: Claude عبر `@anthropic-ai/sdk` بتحميل كسول، Gemini عبر `@google/genai`، ومزود محلي حتمي مضمون التوافر يعمل بلا مفاتيح. الاختبارات عبر Vitest + supertest تغطي المصادقة والتحقق والبوابة وحل المزود — غطاء موجز لكنه في المكان الصحيح.

```
Cybersecurity-AI-Agent/
├─ server.ts          1,513 سطرًا مصدرًا ~8,150
├─ src/
│  ├─ App.tsx         102 الخادم كاملًا في ملف واحد
│  ├─ types.ts        321
│  ├─ lib/api.ts       29
│  ├─ server/llm/      336
│  ├─ data/            752
│  └─ components/     4,896
├─ test/              141
├─ docs/roadmap-report.html 372
└─ bun.lock · vite.config.ts · tsconfig.json · .env.example

rtl/ltr      102
نموذج الحالات الموثق 321
المركزي apiFetch عميل 29
(cclaude/gemini/local) طبقة المزودين 336
cyberData + agentPlatformData 752
وحدات واجهة (859 أكبرها) 10 4,896
api.test.ts + llm.test.ts 141
وثيقة خارطة طريق سابقة 372
```

الطبقة	المكونات	ملاحظة تقييمية
الواجهة	React 19 + Tailwind 4 + Lucide + react-markdown	RTL مُدار من الجذر بشكل صحيح، بلا مكتبة إدارة حالة
الخادم	Express 4 + express-rate-limit + dotenv	تحقق جيد ومعدلات متدرجة؛ البنية تحتاج فصلًا
الذكاء الاصطناعي	طبقة LLM الموحدة + استدعاء Gemini مباشر في الخادم	التجريد ممتاز لكن مسارين يلتقيان جزئيًا (قسم 3)
البيانات	مخازن داخل الذاكرة + ملفا بيانات ثابتة	خُدود موثقة في README لكنها عائق للتقدم التعليمي
الجودة	Vitest + supertest + <code>tsc --noEmit</code>	غياب ESLint يُبقي مستوردات ميتة وأنماطًا غير آمنة

تناقضات إصدارات طفيفة لكنها تلاحظ: تذييل الواجهة يعرض «Core v3.0» و«Gemini 3.7 Flash Engine» بينما `package.json` يقول `0.1.0` والخادم يقصر إلى `gemini-2.5-flash`. توحيد مصدر حقيقة واحدة لهذه القيم (متغيرات بيئة تُقرأ في مكان واحد) يمنع انزلاق المصادقية أمام مستخدم على دراية.

SECTION 02

نقاط القوة الملموسة

قبل تفصيل الملاحظات، من الإنصاف والمنهجية توثيق ما يُحسب للمشروع صراحة. لأن هذه المكونات تشكل «العمود الفقري» الذي أوصي بالبناء عليه وليس استبداله. النقاط الست التالية مرتبة بحسب قيمتها المعمارية، وكل واحدة منها مرفقة بمرجع الملف الدقيق لتسهيل العودة إليها أثناء التطبيق.

1 — تجريد LLM متعدد المزودين نظيف

واجهة `LLMProvider` واحدة بثلاثة تطبيقات، تحميل كسول لـ SDK بحيث لا يتعطل البناء عند غياب حزمة Claude، تدهور رشيق مضمون إلى مزود محلي حتمي، ورمز حالة تشخيصي بلا أسرار. هذا مستوى تصميم يُدرّس، ويجعل إضافة مزود ثالث (Grok أو Llama عبر OpenAI-compatible) مسألة ملف واحد.

`,claudeProvider,geminiProvider,localProvider}.ts`

2 — تحقق مدخلات منوجي وليس استثناءً

الدالة `validateStringField` تُطبق على كل نقطة نهاية تقريبًا: نوع، فراغ، حدود طول صريحة لكل حقل (4,000 للأوامر/الكود، 500 للأهداف). نمط موحد يعيد `VALIDATION_ERROR` بوصف دقيق، الاستثناء الوحيد: حقن `params` في `/api/tools/execute` دون فحص عمق/حجم.

`server.ts:22-52` ومواقع الاستدعاء (13 موضعًا)

3 — تحديد معدل متدرج بوعي تكلفة

طبقتان: عالمي 60 طلبًا/15 دقيقة لكل IP على `/api/*`، وصارم 15/15 على مسارات الذكاء الاصطناعي وحدها، مع رسالة 429 تعيد `retryAfterSeconds` محسوبة. هذا حماية حقيقية لحصة API مدفوعة — وعي ناضج بمخاطر DoS الاقتصادية لا التقنية فقط.

server.ts:636-710

4 — حدود حقن أوامر لنماذج الذكاء

كل مدخل غير موثوق يُغلف بـ `<user_input>...</user_input>` مع تعليمة نظام ثابتة `INJECTION_GUARD` تعامل المحتوى المحصور كبيانات لا تعليمات. الاختبارات تغطي التغليف نفسه. الممارسة مثالية تعليميًا لأنها تحاكي دفاعات الإنتاج الفعلية.

src/server/llm/types.ts:62-72 + اختبار
llm.test.ts:56-60

5 — نموذج مجالات موثق بالأنواع

`types.ts` يعرف 321 سطرًا تغطي دورة التقييم كاملة: `SecurityFinding` بدرجة CVE و CVSS و OWASP وأدلة إثبات وتحقق سلوك إيجابي كاذب وترقيع كود وإعداد، إضافة إلى خطط التنسيق وخطواتها العشر وأحداث الاستعادة. هذا النموذج جاهز ليكون مصدر الحقيقة لمنظومة التقارير المقترحة في القسمين 6-7.

src/types.ts:71-140 (SecurityFinding + OrchestrationPlan)

6 — عادات هندسية صحية

عميل `apiFetch` مركزي يرفق مفتاح الوصول بمكان واحد، ترخيصات ترخيص Apache-2.0 مع شرح في كل ملف، وضع `NODE_ENV=test` يعزل الخادم عن خدمة الوصول لتشغيل الاختبارات، README صادق يوثق حدود الحالة داخل الذاكرة وصغر قيمة المفتاح المدمج في المتصفح — الشفافية جزء من الجودة.

src/lib/api.ts + server.ts:1480-1494 +
README:74-76

SECTION 03

التحليل المعماري العميق — طبقة الخادم

3.1 — المونوليث: ملف واحد يقوم بكل شيء

ملف `server.ts` يضم في وحدة واحدة: دالة تحقق عامة، أربعة مخازن حالة داخلية، محرك تشخيص أخطاء بستة تصنيفات ثنائية اللغة، مولد خطة تنسيق وكلاء كامل بنتائج ثغرات محاكاة، ميدلوير المصادقة والحد من المعدل، ثم 14 نقطة نهاية تتراوح بين مشاريع وبوابة وتنسيق وأدوات واستعادة ودردشة وتدقيق وتقارير. النتيجة العملية: أي تعديل بسيط — كإضافة حقل لتقرير — يتطلب فهم السياق الكامل للملف، ولا يمكن اختبار بوابة الأمان بمعزل عن Express، ويتعارض الملف مع مبدأ المسؤولية الواحدة الذي تلزم به بقية المشروع (طبقة LLM مثال مضاد ممتاز).

المقترح ليس إعادة كتابة بل **تقسيم ميكانيكي يحفظ السلوك**، ويمكن تنفيذه على مراحل مع بقاء الاختبارات الحالية خضراء بعد كل خطوة:

```
// الهيكلية المستهدفة - تقسيم يحفظ السلوك، لا إعادة كتابة
server/
├─ app.ts                خدمة الأصول + trust proxy + Express تركيب
├─ middleware/
│   └─ auth.ts           مقارنة زمنية-ثابتة لاحقًا (x-api-key)
│   └─ rateLimiters.ts   العالمي 60/15 + الصارم 15/15
├─ routes/
│   └─ projects.ts · gateway.ts · orchestrator.ts
│   └─ tools.ts · errorRecovery.ts · geminiRoutes.ts · logs.ts
├─ services/
│   └─ securityGateway.ts دقيقة CIDR ترقية لمطابقة +
│   └─ orchestratorPlan.ts المولد المحكي الحالي كما هو
│   └─ recoveryEngine.ts التشخيص + قواطع الدائرة
├─ stores/
│   └─ types.ts           P2 في SQLite ، الآن Memory تنفيذ → StateStore واجهة
└─ validators/strings.ts لاحقًا zod (أو validateStringField)
```

3.2 — مساران متوازيان للذكاء الاصطناعي (الأهم معماريًا)

المفارقة الأبرز في المستودع: طبقة `src/server/llm` بُنيت بعناية في «المرحلة الأولى» بحسب التوثيق، لكنها موصولة فعليًا بمسار واحد فقط (`/api/gemini/chat`)، بينما يستدعي منسق الوكلاء وتدقيق الكود عميل Gemini مباشرًا عبر دالة `getAIClient()` داخل الخادم — عميل ثانٍ منفصل عن `GeminiProvider`. هذا يعني أن **كل** استدعاء تنسيق/تدقيق يفقد التدهور إلى Claude عند فشل Gemini، ويستحيل اختباره بمعزل عن Google SDK، ويضاعف نقاط الضبط (مفتاحان، Model id في مكانين). الكود التالي يلخص الوضع كما هو:

```
// (و 1301-1343 806-892 server.ts) المسار 1 - يتجاوز الطبقة الموحدة
const ai = getAIClient(); // مكرر داخل الخادم Gemini عميل
const response = await ai.models.generateContent({ model: GEMINI_MODEL, ... });
const parsed = JSON.parse(response.text || "{}"); // فشل الصياغة = سقوط يدوي

// (server.ts:1243-1246) المسار 2 - عبر الطبقة الموحدة ✓
const result = await llmGenerate(chatMessages, { system, temperature: 0.4 });
// claude → gemini → local تدهور مضمون
```

التوصية: حوّل المنسق والتدقيق إلى `generateJSON()` الموجودة أصلًا في الطبقة الموحدة (بتميرير وصف المخطط الحالي كـ `schemaHint`)، ثم احذف `getAIClient()` والمتغير `aiClient` بالكامل. الريح: مزودان حقيقيان في كل المسارات، وإزالة ~120 سطرًا من الأنماط المكررة، وإمكانية محاكاة الردود في الاختبارات عبر حقن مزود وهمي.

3.3 — الحالة داخل الذاكرة: حد موثوق لكنه عائق تعليمي

كل الحالة — المشاريع، سجلات التدقيق (سقف 200)، أحداث الاستعادة (سقف 50)، قواطع الدائرة — تُولد برمجيًا وتنتهي مع العملية. التوثيق صادق بشأن ذلك ويحذر من تعدد النسخ، لكن الأثر التعليمي أعمق من التنبيه: لا يمكن لمتعلم العودة إلى مسوده، ولا يبقى أثر لخطأ ارتكبه أمس، ولا يمكن بناء «ملف متعلم» أو تصدير تقرير من مهمة سابقة. الحل الموصى به ليس قاعدة بيانات كاملة من اليوم الأول، بل واجهة `StateStore` رشيقة (`get/list/upsert/append`) يطبقها مخزن الذاكرة الآن ثم تنفيذ SQLite عبر Prisma لاحقًا دون لمس نقاط النهاية.

3.4 — هندسة المحاكاة أولاً: قوة تعليمية بشرط الشفافية

المنصة تحاكي كل شيء بطبقة خدمات حتمية: `/api/tools/execute` يعيد دائماً `SUCCESS` بمخرجات قلبية، ومولد خطة التنسيق يجد في كل مرة «SQLi OR '1'='1» و«مجلد backup/ مكشوف» بصرف النظر عن وصف المهمة، والمحاكي الطرفي يعيد نتيجة nmap نفسها لأي أمر يبدأ بها. هذه محاكاة تعليمية مشروعة، لكن الخطر يكمن في الطرح: الواجهة تعرض النتائج بلغة الإنجاز («تم إصدار التقرير النهائي...»، «Agent is assembling formal executive report...») مما قد يرسخ لدى المتعلم انطباعاً بأن هناك فحصاً فعلياً يحدث. توصيتي مزدوجة:

R1 وسم واضح لوضع المحاكاة

شارة «SIMULATION» ثابتة على النتائج المحاكاة في الواجهة، وسطر توضيحي في مخرجات الخادم — لئلا يخطئ المبتدئ بين محاكاة آمنة وفحص حقيقي.

R2 واجهة Executor قابلة للتبديل

تعريف `ToolExecutor { run(toolId, target, params) }` بتطبيقين: `SimulatedExecutor` (الحالي) و `DockerExecutor` مستقبلياً لتشغيل أدوات حقيقية داخل حاوية معزولة على هدف مختبر — فتنحول المنصة من بيئة عرض إلى بيئة تمرين فعلية دون تغيير نقاط النهاية.

3.5 — بوابة الأمان: المطابقة بالسلاسل الجزئية قابلة للتجاوز

بوابة التفويض مفهومها ممتاز، لكن تطبيقها يعتمد على `includes()` النصية في الاتجاهين (السماح والمنع)، وهذا يؤدّ ثلاث فئات من العيوب يمكن للمتعمّل الفضولي اكتشافها وتجربتها فوراً:

```
// server.ts:369-373 — المنح ضمن النطاق بالمطابقة الجزئية
const isInScope = project.inScope.some(a => targetClean.includes(a.replace("*.","")))
|| targetClean.includes("192.168.") || targetClean.includes("lab") ||
targetClean.includes("local");

// "lab" يجتاز لأنه يحوي "evil-lab.attacker.com" : تجاوز فعلي
// النطاق كجزئية IP يجتاز لأنه يحوي "evil.com". تجاوز فعلي: "192.168.1.50"
// (includes جزئي) "يحوي" "192.168.1.1" فُرض "192.168.1.10" outOfScope : إنكار خاطئ
```

المعالجة ثلاثية: مطابقة IP عبر مكتبة CIDR مثل `ipaddr.js` (تحليل ثم احتواء شبكة)، ومطابقة مضيفين بالنطاق الدقيق بعد إزالة نقطة النهاية بدل الاحتواء، وحذف «الهروب الشامل» الذي يسمح لأي هدف يحوي كلمة `lab` أو `local`. من الناحية التعليمية أقترح إبقاء نسخة «الثغرة» موثقة كدرس قائمة بجانب النسخة المصححة — فتحليل سبب قابلية التجاوز هو بالضبط نوع التعلم الذي ضمنت المنصة لأجله.

3.6 — ملاحظات خادم إضافية موثقة

الموضوع	الملاحظة	الأثر
server.ts:633	سقف جسم الطلب 15MB بينما أكبر حقل مقبول 4,000 حرف	سطح DoS ذاكرة بلا فائدة وظيفية: 1MB كافٍ جدًا
عام	لا Helmet ولا ترويسات أمن ولا سياسة CORS معلنة	غياب X-Content-Type-Options/HSTS في الإنتاج

الموضوع	الملاحظة	الأثر
server.ts:695	مقارنة مفتاح الوصول بـ <code>≠</code> غير الزمنية-الثابتة	هامشي هنا، لكن استبدالها بـ <code>timingSafeEqual</code> عادة يستحق التعليم
server.ts:1034	قيمة <code>successRatePercentage: 98.4</code> ثابتة تُعرض كمؤشر حي	يُضعف الثقة بمؤشرات حقيقية أخرى كعدد الاستعدادات
server.ts:1063-1129	مسار الذكاء الاصطناعي للاستعادة لا يحدث قواطع الدائرة بعكس المسار المحلي	تتناقض حالة يظهر في شبكة القواطع بعد تشخيصات AI
server.ts:897 وغيرها	معرفات <code>plan_\${Date.now()}</code> قابلة للاستخدام تحت تزامن	استبدالها بـ <code>crypto.randomUUID()</code> أكثر أمانًا وتعليقًا
server.ts:808-830	نصوص عربية لغة-قوالب مدمجة في منطق الخادم	نقلها لطبقة قوالب تسهّل إضافة لغات وتراجع الصياغة

SECTION 04

التحليل المعماري العميق — الواجهة الأمامية

4.1 — ثلاثة اصطلاحات تحويل متعارضة بلا طبقة موحدة

اللغة تُمرر كخاصية `'ar' | 'en'` من `App.tsx` إلى كل مكون، ثم يتعامل كل مكون معها بطريقة: مئات الثلاثيات المضمنة `{isAr ? '...' : '...'}` في `JSX`، وحقول متوازية `titleAr/titleEn` في المصفوفات المحلية، وخرائط `labelAr/labelEn` في أخرى. الأسوأ أن بعض الحقول عربية-فقط وتُعرض للغتين معًا: حقل `impact/prevention` في بنود OWASP وأوصاف تقنيات MITRE في `cyberData.ts` تُرسم كما هي في وضع الإنجليزية. الاستثمار في قاموس واحد `t(key)` أو مساعد `{ar, en}` ينهار هذا التنوع إلى اصطلاح واحد قابل للتدقيق التالي.

4.2 — أخطاء وظيفية مؤكدة (تحقق يدوي)

خطأ الربط CTF → الوكيل: خاصية `onAskAgentAboutChallenge` التي يمررها `CtfArena` مع السيناريو كاملاً يُهمل الوسيط في `App.tsx:43-45` وينتقل إلى تبويب `orchestrator` بدل `agent` — زر «اسأل المرشد الذكي» عن التحدي لا يفعل المطلوب. الإصحاح سطر واحد: `setActiveTab('agent')` مع تخزين السيناريو في حالة مشتركة.

تحدي 02-CTF غير قابل للحل (تم التحقق بالفك): السلسلة السداسية في نص السيناريو `526b7842...7d` تفكك إلى `RkxBR05vbmVyc2VjcnV0eXNkYW5jZTIwMjYy}` وهي Base64 غير صالحة (خطأ padding) ولا تفكك إلى العلم المعلن `FLAG{never_rely_on_encoding_for_security_2026}`؛ التلميح الثاني يقول إن الناتج يبدأ بـ `RkxCU...` وهو أيضًا خطأ (الناتج الفعلي يبدأ `RkxBR...`)، بينما يذكر «شرح الحل» سلسلة Base64 صحيحة مختلفة عن ما تنتجه البيانات فعليًا — أي أن المشكلة في الـ hex وليست في شرحه. الإصحاح: استبدال السلسلة السداسية بترميز السلسلة الصحيحة: `526B7842523374755A585A6C636C39795A57783558323975583256755932396B6157356E58325A76636C39.7A5A574E31636D6C30655638794D44493266513D3D`

4.3 — أعلام التحديات داخل حزمة العميل

قيم `flag` وشروحات الحل لكافة التحديات تُشحن نصًا صريحًا في حزمة `cyberData.ts` (JavaScript) والتحقق مجرد مقارنة سلسلة في المتصفح، فيمكن استخراج الأعلام كلها من أدوات المطور في ثوانٍ — وهو نمط يتعارض مع درس أمني تعزز المنصة إيصاله. الحل الموصى به: نقطة نهاية صغيرة `POST /api/ctf/submit` `{scenarioId, flag}` تتحقق خادميًا وتسجل التقدم في المخزن الدائم عند وصوله، مع إبقاء التلميحات في العميل. بمجرد وصول طبقة التخزين (P2) تُفتح مجانًا ميزات «أعلامي/لوحة تقدم».

4.4 — كود ميت وتكرار أنماط

قراءة 40% من `agentPlatformData.ts` لا يستهلكه أي مكون: `TOOL_PLUGINS` (150 سطرًا من تعريفات أدوات موثقة) و `LAB_ENVIRONMENTS` (بيئات مختبرات مع `docker-compose hints` وأعلام) وحقلاً `systemPrompt/capabilities` في `SYSTEM_AGENTS`. هذا ليس فائضًا للقص بل **رأس مال مؤجل**: تعريفات الأدوات هي بالضبط ما يحتاجه مُشغّل DockerExecutor المستقبلي، وبيئات المختبرات هي بوابة المختبرات الحقيقية في القسم 8. الأفضل تفعيلها تدريجيًا بدل حذفها. أما التكرار الي هو أكثر إلحاحًا:

النمط المتكرر	التكرار	المعالجة المقترحة
نسخ للحافظة + مؤقت إعادة حالة 2 ثانية	6 مرات	خطاف <code>useCopy()</code> واحد مع تنظيف المؤقت عند إلغاء التركيب (المؤقتات حاليًا بلا cleanup)
خرائط ألوان الشدة/الصعوبة	5 اصطلاحات	موديول <code>severity.ts</code> موحّد (فئة + نص + لون) يستهلكه الجميع
نماذج جلب يدوية (method/headers/body + فحص ok)	كل مكون	طبقة <code>api/</code> موصوفة بالأنواع — لا حاجة لتكرار Content-Type فأصله موجود في <code>apiFetch</code>
شريط تبويب داخلي بنفس الأصناف الشريطية	+5 مكونات	مكون <code><SegmentedTabs></code> مشترك مع <code>role="tablist"</code>
تنفيذ MD5 يدوي بطول 150 سطرًا داخل جسم المكون	CyberToolbox: 49	نقله لدالة موديولية خالصة (إعداد إنشاؤه حاليًا في كل رسم) أو Web Worker

4.5 — إمكانية الوصول وRTL

التبديل على مستوى المستند مضبوط جيدًا (`document.documentElement.dir/lang`) في `App.tsx` مع افتراضي RTL في `index.html`، لكن داخل المكونات الصفر التام لخصائص ARIA في كامل الشجرة: أشرطة التبويب أزرار بلا `role="tab"/aria-selected`، والأزرار الأيقونية (نسخ/مسح/تشغيل) تعتمد `title` وحدهم، والحالات تُنقل

باللون ووحده بلا `aria-live` للتأنيح غير المتزامنة. في RTL تحديدا تظهر مشكلة ثانية: خصائص فيزيائية مثل `absolute left-2.5 + pl-8` في حقول البحث تجعل عدسة التكبير تقع في الجهة الخطأ عند العربية، وتُصلح بدائل منطقية (`start-2.5/ps-8`). إضافة `ErrorBoundary` واحد حول التبويبات ستحول انهيار أي مكون من «شاشة بيضاء كاملة» إلى رسالة معزولة — إصلاح منخفض الكلفة عال القيمة لمنصة تتعامل مع ردود ذكاء اصطناعي متغيرة البنية.

الموضوع	الملاحظة	الأولوية
App.tsx:43-45	ربط CTF → وكيل يتجاهل السيناريو ويذهب لتبويب خاطئ	P0
cyberData.ts:294-313	hex: CTF-02 لا يفكك إلى العلم (تحدي معطل)	P0
تسع واجهات	فقدان حالة الشات/الطرفية/التقدم عند تبديل التبويب (إلغاء تركيب) ولا persistence للغة والتقدم	P1
App.tsx:59-73	لا ErrorBoundary — انهيار مكون واحد يبيض التطبيق كله	P0
6 مواضع نسخ	<code>setTimeout</code> بلا تنظيف عند إلغاء التركيب + clipboard بلا try/catch	P1
cyberData.ts:124-221	قاعدة OWASP تعرض «Top 10» وتضم 5 بنود فقط (A01-A05) بيانات 2021	P1
كل الشجرة	صفر aria/role؛ حقول بحث بخصائص فيزيائية تنعكس خطأ في RTL	P2
CyberToolbox:202,228	استخدام <code>escape/unescape</code> المهجورة	P2

SECTION 05

توصيات التحسين المرتبة بالأولوية

الجدول التالي يدمج كل ملاحظات التقرير في خطة تنفيذ واحدة قابلة للتتبع. الأولويات محكومة بقاعدتين: P0 ما يستعيد وظائف معطلة أو يغلق مخاطر فعلية بأقل جهد، ثم ما يفتح الطريق لطبقات البناء الأعلى (تخزين دائم، تقارير، مخبرات حقيقية). كل بند مرفق بموضعه التقريبي والفائدة التعليمية المباشرة منه حتى تبقى خطة التطوير نفسها مادة تعلم.

الطبقة	بند العمل	الملفات المتأثرة	القيمة
P0	إصلاح ربط «اسأل المرشد» بالتحديات (تخزين السيناريو المختار والانتقال لتبويب الوكيل)	App.tsx , CtfArena.tsx	مسار تعلم رئيسي يعمل
P0	تصحيح سلسلة CTF-02 السداسية إلى ترميز العلم الصحيح	cyberData.ts	تحدي قابل للحل فعليًا

الطبقة	بند العمل	الملفات المتأثرة	القيمة
P0	خفض سقف الجسم إلى 1MB + ErrorBoundary حول التبويبات + try/catch للحافطة	server.ts, App.tsx	مئات بجهد ساعات
P0	وسم «SIMULATION» على النتائج المحاكاة (تنسيق/ أدوات/طرفية)	3 مكونات عرض	نزاهة تعليمية
P1	تقسيم server.ts وفق هيكلية القسم 3.1 مع إبقاء الاختبارات خضراء	server.ts → */server	بوابة كل التحسينات اللاحقة
P1	توحيد مسار الذكاء الاصطناعي عبر generateJSON وحذف getAIClient	server.ts, */llm	مزودان في كل المسارات
P1	ترقية بوابة النطاق: js.ipaddr للـ CIDR ومطابقة مضيف دقيقة وإلغاء هروب lab/local	securityGateway	مفهوم التفويض يصبح صحيحًا
P1	قاموس t(key) مودد + استكمال OWASP إلى 10 بنود وربط حقول عربية-فقط	src/i18n, cyberData	محتوى ثنائي اللغة مكتمل
P1	useCopy + severity.ts + SegmentedTabs + إضافة Helmet وثرويسات الأمان	src/lib, server	إخمداد +20 تكرارًا
P2	StateStore بتنفيذ SQLite/Prisma للمشاريع والسجلات والنتائج والأعلام	stores/*, schema.prisma	تقدم دائم وملف متعلم
P2	تحقق خادمي للأعلام CTF مع تسجيل التقدم ومنع استخراج الأعلام من الحزمة	ctfRoutes, CtfArena	نزاهة التحديات
P2	منظومة التقارير وفق القسمين 6-7 (قالب، PDF متجهي، ربط خط النتائج)	report/*, ReportBuilder	المحور الأهم لديك
P2	واجهة ToolExecutor تحضر DockerExecutor المستقبلي + تفعيل TOOL_PLUGINS	services/execut ors	من المحاكاة إلى التمرين
P3	ملف متعلم: مسارات صعوبة، XP وشارات، تصدير تقرير تقدم (يستولك منظومة التقارير)	*/profile	تعلم متصل ومحقق
P3	WebSocket لسجلات حية بدل مصفوفات ثابتة + نظام أدوار ARIA شامل	ws/*, components	تفاعل واقعي وشامل
P3	مختبرات حقيقية معزولة: تفعيل docker-compose (Juice مع LAB_ENVIRONMENTS Shop/DVWA)	*/labs	تمارين فعالية آمنة

منظومة توليد التقارير — الوضع الحالي والفجوات

وحدة التقارير هي الأقرب لطالبك («تقارير شاملة تمكّني من التطوير») ولذلك قُصّلت في قسمين. سلوك النظام الحالي محسوم من الكود: الواجهة تجمع بيانات النموذج وقائمة الثغرات، ثم يستقبل الخادم الطلب في

يستدعي أي نموذج ذكاء اصطناعي، تُعرضه الواجهة عبر react-markdown مع زر نسخ وزر تنزيل [.md](#). الغريب الإيجابي أن نموذج البيانات الغني موجود كاملًا في [SecurityFinding](#) (تولّد نسخ منه في كل مهمة تنسيق!) بينما تستخدم وحدة التقارير نموذجًا مبسطًا [SecurityReportFinding](#) يهدر خمسة من أهم الحقول.

```
// خط الأنايب الحالي - يلاحظ منه
UI findings (SecurityReportFinding مبسط) → POST /api/gemini/generate-report
  → (فقط title/severity/cwe :جدول واحد لكل الثغرات) قالب حرفي ثابت
  → لا رسوم · لا نطاق/منهجية كاملة · لا DOCX · لا PDF · لا .md تنزيل + ReactMarkdown

// (تولّد فعلاً في كل مهمة) SecurityFinding النروة المهمة في
cvssScore · evidence · validation.falsePositiveAnalysis
remediation.{codeFix, configPatch, hardeningSteps, verificationCommand} · owaspCategory
```

الفجوات الثماني المحددة

61 لا مخرجات مؤسسية

— تنزيل Markdown وحده؛ لا PDF متجهي ولا DOCX قابل للتعديل، وكلاهما متوقع في أي سيناريو تقييم حقيقي.

62 قالب أعمى بالهيكل

— نص القالب ثابت في الخادم: الملخص التنفيذي وخارطة المعالجة نصوص مقولة مسبقًا لا تتأثر بالمحتوى، scopeg المستلم لا يُستخدم في الإخراج.

63 تهريب ثراء البيانات

— CVSS والأدلة codeFixg configPatchg وخطوات التحسين كلها متاحة في نموذج المهمة ولا تصل للتقرير.

64 لا ارتباط بوقائع المهمة

— نتائج Orchestrator المولدة (generatedFindings) تعيش في حالة المكون وتختفي عند تبديل التبويب، ولا تصل لوحدة التقارير تلقائيًا.

65 لا تمثيل بصري

— لا توزيع شدة ولا نسب تحقق ولا مصفوفة OWASP؛ المدراء يقرؤون الجداول أفضل من الشارات.

66 لا تمييز جمهور

— الملخص التنفيذي والتفصيل التقني ينصهران في قالب واحد؛ التقارير الجادة تفصل القسمين بمستويات تفصيل مختلفة.

67 لا هوية وطباعة

— لا غلاف، لا ترويسة/تذييل، لا ترقيم صفحات، لا تصنيف سرية، ولا معلومات منهجية كاملة (المنهجية مذكورة نضًا فقط).

68 لا مسودات ولا إصدارات

— التقرير حالة مؤقتة في الذاكرة؛ لا حفظ ولا رقم إصدار ولا سجل تعديلات.

المكتبات الموصى بها للتكامل الداخلي

المعايير الحاكمة للاختيار هنا: **دعم العربية/RTL أصلاً لا كحيلة** (تشكيل الحروف واتجاه الفقرات)، **وجودة الإخراج** بما يليق بتقرير يُسلّم لجهة، و**كلفة التشغيل** على خادم صغير، و**التوافق** مع stack قائم (+ Express React). الجدول التالي يرشح المرشحين بصرامة كاملة، ثم يأتي مقترح الطريق المركب.

المكتبة	الدور	دعم العربية/RTL	متى تختارها
Handlebars / EJS	محرك قوالب HTML	ممتاز — HTML أصلاً يدير الاتجاه بـ dir/css	الأساس قبل أي مولد PDF: فصل القالب عن المنطق، قالبان (عربي/ إنجليزي) من مصدر بيانات واحد
Puppeteer (خادمي)	PDF متجوي من HTML	ممتاز — محرك Chromium يشكّل العربية ويدعم @page/ترويسة/ترقيم	الخيار الأول لجودة الإنتاج؛ التكلفة تثبت Chromium على الخادم (~300MB) دون أي عبء على متصفح المستخدم
react-pdf-renderer	PDF برمجي من مكونات React	جزئي — يدعم direction/rtl مع تسجيل خط عربي، لكن الجداول والتفاصيل الدقيقة تقريبية	تقارير برمجية بمكوّنات موحدة الهوية بلا قوالب HTML وبلا متصفح
html2pdf.js (html2canvas)	PDF من DOM في المتصفح	جيد بصرياً (يلقط RTL كما يعرضه المتصفح)	تصدير سريع للمعاينة فقط — الناتج نص صوري غير قابل للتحديد/البحث
jsPDF + autotable	PDF برمجي عميل	ضعيف للعربية — لا تشكيل حروف ولا bidi أصيل	تيارات إنجليزية فقط؛ لا أنصح به لمخرجات عربية كثيفة النص
docx (npm)	DOCX قابل للتعديل	جيد — يدعم bidi على مستوى القسم والفقرة والجداول	عندما يطلب «العميل» ملفاً يعدّله فريقه — مكمل وليس بديلاً عن PDF
Recharts / Chart.js	رسوم التقرير (توزيع شدة، نسب تحقق)	جيد مع خط عربي مناسب في التسميات	لوحات تنفيذية داخل القالب قبل التصدير
markdown-it + highlight.js	طبقة تحرير مخرجات LLM	جيد	إذا لاحقاً صار التقرير نصاً مولداً؛ تنقية ثم حقنه في القالب بدل Markdown الخام

المرحلة 2 — أسبوعان: PDF متجهي خادمي

نقطة نهاية `GET /api/reports/:id/pdf` تعيد استخدام القالب نفسه عبر Puppeteer مع خط عربي مضقن و `printBackground` وترقيم `page@` — تحليل G1 كاملة بجودة طباعة فعلية، وتغقل G8 بتخزين المسودات في `StateStore` مع رقم إصدار. القالب واحد يخدم الويب والطباعة عبر ورقة أنماط `print`.

المرحلة 1 — أسبوع: قالب Handlebars +

طباعة المتصفح

مخطط بيانات JSON موحد يملأ فجوات G2/G3 (كل حقول `SecurityFinding` تصل التقرير)، قالب `report-` `ar.hbs` كامل الأقسام (غلاف، ملخص تنفيذي، مصفوفة نتائج، تفاصيل لكل نتيجة بأدلتها وترقيعها، منهجية ونطاق)، وزر «طباعة/PDF» عبر `print CSS`. RTL مثالي فورًا وبدون أي اعتماد خادمي جديد، ويحل G1 جزئيًا و G6/G7 كليًا.

لمسات الجودة التي تفرق التقرير المؤسسي

تصنيف سرية في الغلاف والتذييل، جدول تحكم بالإصدارات، معرف تقرير ومعرف تتبع مهمة من `traceId` الموجود أصلًا، صفحة منهجية تسرد `WSTG/PTES/NIST` مع حدود الاختبار، وقسم «إعادة الاختبار» يستهلك حقل `retestStatus` الموجود في النموذج. تفاصيل صغيرة تصنع مصداقية المخرج التعليمي كتدريب حقيقي.

المرحلة 3 — أسبوعان: DOCX + رسوم + ربط

خط النتائج

مخرج `docx` موازي من نفس المخطط (نفس البيانات، قالب ثانٍ)، و `Recharts` للتوزيع الشددي داخل القالب (G5)، وربط زر «استيراد نتائج المهمة» ينقل `generatedFindings` من `Orchestrator` إلى `ReportBuilder` بدل الإدخال اليدوي (G4). عندها تكتمل منظومة: مهمة محاكاة → نتائج → تقرير PDF/DOCX موقع بمنهجية ونطاق.

```
// (Express + Handlebars + Puppeteer) مسار المرحلة 2 - جوهر التنفيذ
import puppeteer from 'puppeteer';
import Handlebars from 'handlebars';

app.get('/api/reports/:id/pdf', async (req, res) => {
  const data = await reportStore.get(req.params.id); // من StateStore
  const html = tplAr({ ...data, t: dict[req.query.lang ?? 'ar'] });
  const browser = await puppeteer.launch({ args: ['--no-sandbox'] });
  const page = await browser.newPage();
  await page.setContent(html, { waitUntil: 'networkidle0' }); // انتظار الخطوط
  const pdf = await page.pdf({ format: 'A4', printBackground: true,
    displayHeaderFooter: true });
  await browser.close();
  res.type('application/pdf').send(pdf);
});
```

دليل عملي يدعم التوصية: هذا التقرير الذي تقرأه الآن هو بالضبط الطريق المقترح — قالب HTML عربي RTL مع خط Cairo ثم تحويل خادمي عبر Chromium إلى PDF متجهي قابل للتحديد والبحث. أي أن أسلوب الإخراج المستهدف لمنصتك مجرب هنا على وثيقة بنفس لغتك واتجاهك.

خارطة الطريق التنفيذية وتعزيز القيمة التعليمية

خارطة زمنية بأربع مراحل ومخرجات قابلة للقياس

المرحلة	النطاق	مخرجات القبول المقترحة
P0 • أسبوع واحد	السريعات الأربعة	زر «أسأل المرشد» يفتح محادثة تحمل سياق التحدي؛ CTF-02 يُحل بالسلسلة الصحيحة من داخل المنصة؛ سقف الجسم 1MB؛ ErrorBoundary يعرض رسالة معزولة عند رمي خطأ متعمد؛ شارة SIMULATION ظاهرة
P1 • أسبوعان	إعادة الهيكلة والتوحيد	شجرة /server* بلا ملف يتجاوز 300 سطر؛ مسار واحد للذكاء الاصطناعي عبر الطبقة الموحدة (استيراد getClient = صفر)؛ اختبار بوابة يثبت رفض evil-lab.attacker.com وقبول 192.168.1.10؛ قاموس i18n يغطي 100% من النصوص
P2 • أسبوع	الاستمرارية والتقارير	StateStore/Prisma يبيقي المشاريع والنتائج بعد إعادة التشغيل؛ تحقق خادمي للأعلام مع إخفاؤها من الحزمة؛ نقطة نهاية PDF متجهية تُخرج تقريرًا بغلاف وترويسة وترقيم ورسوم توزيع شدة
P3 • شهر فأكثر	التعلم المتقدم	ملف متعلم بمسارات وXP وشارات يصمد عبر الجلسات؛ WebSocket يجعل سجلات التنسيق حية فعليًا؛ DockerExecutor يشغل مختبر Juice Shop معزولًا ضمن نطاق مشروع مخصص

أفكار تعزز دعم التعلم والتفاعل (هدفك المعلن)

E1 ملف المتعلم أولاً

— بمجرد وصول التخزين الدائم: مستوى لكل مجال (ويب/شبكات/تشفير/جنائي)، XP من التحديات والتقارير المنجزة، وشارات (أول SQLi مفسرة، تقرير مكتمل، استعادة خطأ...). التقدم المرئي هو أقوى محفز تعليمي موثق.

E2 وضع «اشرح كأنني مبتدئ/خير»

— حل مستوى في دردشة الوكيل يوجه systemInstruction الموجود أصلاً، فيغير عمق الشرح دون بنية جديدة — استثمار صغير بأثر تعليمي كبير.

E3 تقرير تقدم للمتعلم

— منظومة التقارير نفسها تُعاد استخدامها لتوليد «تقرير تقدم شخصي» (تحديات محلولة بمرور الوقت، تغطية OWASP، نقاط ضعف مفاهيمية) — يتحول التقييم من أداة إخراج إلى مرآة تعلم.

E4 مختبرات حقيقية آمنة

— تفعيل LAB_ENVIRONMENTS المبيت مع Juice Shop/DVWA docker-compose داخل شبكة معزولة وهدف مسجل في مشروع مخصص؛ عندها تتحول بوابة النطاق من درس نظري إلى عائق يُلمس فعليًا عند محاولة تجاوز النطاق.

— 10 أسئلة عشوائية من قاعدة المعرفة + تحدي قصير، بتصحيح فوري وربط بنتائج ملف المتعلم — يكمل حلقة التعلم: تعلم ثم تمرين ثم قياس.

خلاصة

CyberGuard AI مشروع ناضج التفكير أكثر من كونه ناضج الهيكلية — وهذا ترتيب صحي تمامًا لمشروع تعليمي في مرحلته. أساسه الأمني (تحقق، حدود معدل، حدود حقن، بوابة تفويض) جاهز للاعتماد، وطبقة المزودين الموحدة نموذج يُحتذى؛ والمطلوب هو استثمار مركّز في ثلاثة اتجاهات متتالية: تقسيم يحفظ السلوك، ثم استمرارية تمنح المتعلم ذاكرة، ثم منظومة تقارير تُحوّل كل تمرين محاكي إلى مخرج مؤسسي قابل للعرض والتحرير. بترتيب الأولويات أعلاه، كل أسبوع عمل يضيف قيمة مستقلة دون أن يقفل الطريق لما بعده، ويبقى المشروع ما صُمم لأجله: بيئة تعلم آمنة تعلّم مفاهيم الأمن بممارسته في آن واحد.

قائمة تحقق سريعة قبل الانطلاق — تسلسل جلسات العمل المقترح

- ☐ **جلسة 0:** إنشاء فرع `refactor/baseline` وثبيت خط الأساس: `npm run test` و `npm run lint` أخضران قبل أي تعديل، مع إضافة ESLint (بما فيها `noUnusedLocals`) لالتقاط الكود الميت آليًا.
- ☐ **جلسة 1 (سريعات P0):** إصلاح `App.tsx:43` وسلسلة CTF-02 ثم التحقق اليدوي من أن التحدي يُحل من داخل المنصة، وإضافة `ErrorBoundary` وشارات `SIMULATION` وخفض سقف الجسم إلى 1MB.
- ☐ **جلسة 2-4:** نقل `server.ts` وحدة وحدة إلى الهيكلية المستهدفة، مع تشغيل الاختبارات بعد كل نقل وإبقائها خضراء — ثم حذف `getAIClient()` وتوصيل المنسق والتدقيق بـ `generateJSON`.
- ☐ **جلسة 5:** ترقية بوابة النطاق بـ `ipaddr.js` وإضافة اختبارين: رفض `evil-lab.attacker.com` وقبول `192.168.1.10` — درس تفويض حي وموثق بالاختبار.
- ☐ **جلسة 6-7:** قاموس `t(key)` واستكمال OWASP Top 10 وإصلاح الحقول العربية-فقط، ثم خطافات `useCopy` `SegmentedTabsg severity.ts`.
- ☐ **جلسة 8+:** (بالتوازي ممكن): هيكلية `report/` مع قالب `report-ar.hbs` وأول PDF متجهي عبر Puppeteer من مخرجات مهمة حقيقية — أول مخرج ملموس لمنظومة التقارير.

بناءً على أساس جيد، الخطوة القادمة هندسية لا مفاهيمية.

ابدأ بسريعات P0 في جلسة واحدة، ثم قسّم الخادم تدريجيًا مع إبقاء الاختبارات خضراء، وتعامل مع منظومة التقارير كخط إنتاج مستقل يبدأ من قالب Handlebars وينتهي بـ PDF متجهوي عربي — وستجد أن كل تحسين معماري هنا يترجم مباشرة إلى قيمة تعليمية أوضح للمتعلم.